

Agent IA

Copiloto personal de gestión de conocimiento, estudio y plataforma
multi-agente

Especificación de Requerimientos de Software (SRS)

Ingeniería de Sistemas

Autor: Miguel Ángel Polo Castro

27 de agosto de 2026

Índice

1. Introducción y marco metodológico	2
2. Nivel 1 — Requerimiento de negocio	2
3. Nivel 2 — Requerimientos de usuario	2
4. Nivel 3 — Requerimientos de software	3
4.1. Requerimientos funcionales	3
4.2. Requerimientos no funcionales	5
5. Diagrama de casos de uso	6
6. Diagrama de clases	7
6.1. Modelo de dominio	7
6.2. Jerarquía de agentes	7
7. Arquitectura de componentes	8
8. Diagrama de secuencia — crear y categorizar una nota	8
9. Flujos e integración de API	9
10. Wireframes	11
11. Decisiones de arquitectura clave	11
12. Conclusiones y estado de implementación	12

1 Introducción y marco metodológico

Agent IA (evolución y formalización del concepto de arquitectura multi-agente) es un copiloto personal de inteligencia artificial cuyo propósito es automatizar la organización, relación y mantenimiento del Segundo Cerebro (Notion + Obsidian) y del espacio de estudio de ingeniería de Miguel, reduciendo el trabajo manual actual mediante agentes de IA especializados y orquestación determinista con Hermes.

Este documento sigue la metodología de ingeniería de requerimientos de tres niveles:

- **Nivel 1 – Negocio:** responde *¿qué se debe hacer y por qué?*
- **Nivel 2 – Usuario:** responde *¿para quién y qué necesita?*
- **Nivel 3 – Software:** responde *¿cómo se construye?* (requerimientos funcionales y no funcionales)

El ciclo de trabajo aplicado en cada nivel es **elicitación** → **análisis** → **especificación** → **validación**, y la priorización de requerimientos utiliza el esquema **DINO** (Deseable, Importante, No implementable, Obligatorio).

Glosario de siglas

Sigla	Significado
RU	Requerimiento de usuario
RF	Requerimiento funcional
RNF	Requerimiento no funcional
PKM	<i>Personal Knowledge Management</i> , gestión de conocimiento personal
ETL	<i>Extract, Transform, Load</i> , pipeline de sincronización de datos
RAG	<i>Retrieval-Augmented Generation</i> , generación aumentada por recuperación vectorial
EDA	Arquitectura orientada a eventos
LLM	Modelo de lenguaje de gran escala
SM-2	Algoritmo clásico de repetición espaciada (usado por Anki)

2 Nivel 1 — Requerimiento de negocio

Construir un copiloto personal de gestión de conocimiento y estudio que automatice la organización, relación y mantenimiento del Segundo Cerebro (Notion + Obsidian) y del espacio de estudio de ingeniería, reduciendo el trabajo manual actual y mejorando la calidad del aprendizaje, mediante agentes de IA especializados que operan sobre esa información en vez de requerir captura y orden manual constante.

Este enunciado es la vara de medida de todo el documento: cualquier requerimiento que no reduzca manualidad ni mejore el aprendizaje debe cuestionarse antes de aceptarse.

3 Nivel 2 — Requerimientos de usuario

Un solo stakeholder: **Miguel**, como estudiante de ingeniería de sistemas y persona.

ID	Descripción	Actor	Prioridad
RU1	El usuario debe poder mantener su Segundo Cerebro (hábitos, notas, objetivos, proyectos, tareas, áreas) sin captura y organización 100 % manual.	Miguel	Alta
RU2	El usuario debe poder gestionar su Study Board de ingeniería vinculado al Segundo Cerebro.	Miguel	Alta
RU3	El usuario debe poder aplicar métodos de estudio (repetición espaciada, active recall, notas Cornell) dentro de ese espacio.	Miguel	Alta
RU4	El usuario debe poder conversar con un asistente que consulte y organice su información personal/académica.	Miguel	Media
RU5	El usuario debe poder delegar tareas a agentes especializados (curaduría, estudio, sincronización, planificación) en vez de uno genérico.	Miguel	Media
RU6	El usuario debe poder gestionar tareas/recordatorios sincronizados entre Todoist, Calendar y su Segundo Cerebro.	Miguel	Alta
RU7	El usuario debe poder acceder al sistema desde su teléfono.	Miguel	Media

Nota de alcance: los módulos de *Finanzas* y el usuario secundario (madre de Miguel), presentes en el proyecto obsoleto, quedan fuera de este alcance por decisión explícita, y podrían retomarse en una fase futura si el sistema demuestra ser escalable (RNF1).

4 Nivel 3 — Requerimientos de software

4.1 Requerimientos funcionales

RU1 — Segundo Cerebro sin manualidad (AgenteCurador)

ID	Descripción	Prioridad
RF1.1	El sistema debe sincronizar automáticamente Notion ↔ Obsidian sin intervención manual (ETL).	Alta
RF1.2	El sistema debe categorizar y relacionar automáticamente notas/tareas nuevas según áreas y <i>topics</i> existentes, usando el LLM local.	Alta
RF1.3	El sistema debe detectar posibles duplicados o notas huérfanas y sugerir su integración.	Media
RF1.4	El sistema debe permitir registrar cumplimiento de hábitos por lenguaje natural (voz/chat) en vez de edición manual de la base de datos.	Media
RF1.5	El sistema debe ejecutar un barrido periódico (cron, vía n8n) del inbox para organizar notas sin procesar.	Alta

RU2 — Study Board (AgenteEstudio)

ID	Descripción	Prioridad
RF2.1	El Study Board debe mantenerse sincronizado y relacionado con el Segundo Cerebro.	Alta
RF2.2	El sistema debe integrarse con Google Drive para vincular documentos académicos al Study Board.	Media
RF2.3	El sistema debe integrarse con Canvas LMS para importar tareas y fechas límite académicas.	Media
RF2.4	El sistema debe preparar y organizar material de estudio listo para subir manualmente a NotebookLM (no automatizable por falta de API pública).	Baja
RF2.5	El sistema debe permitir consultar Gemini como agente externo de estrategia de aprendizaje, de forma explícita, nunca automática por defecto.	Baja

RU3 — Métodos de estudio

ID	Descripción	Prioridad
RF3.1	El sistema debe generar y programar tarjetas de repetición espaciada (algoritmo tipo SM-2) desde las notas del Study Board.	Alta
RF3.2	El sistema debe notificar cuándo hay repasos pendientes.	Alta
RF3.3	El sistema debe ofrecer una plantilla reutilizable de notas Cornell en Notion/Obsidian.	Media
RF3.4	El sistema debe registrar el resultado de cada repaso (acertado/fallado) y ajustar la programación futura.	Alta

RU4 — Asistente conversacional

ID	Descripción	Prioridad
RF4.1	El sistema debe permitir conversar por texto (HUD) y por voz bilingüe ES/EN.	Alta / Media
RF4.2	El asistente debe responder sobre el estado del Segundo Cerebro y del Study Board (pendientes, próximos repasos, notas recientes).	Alta
RF4.3	El asistente debe usar el LLM local por defecto para toda consulta con datos personales.	Alta
RF4.4	El sistema debe ser accesible por Telegram (gateway nativo de Hermes) como punto de entrada móvil.	Media

RU5 — Agentes especializados

ID	Descripción	Prioridad
RF5.1	El sistema debe implementar Hermes como agente orquestador único, que delega en cuatro agentes especializados con dominio de datos separado: Curador, Estudio, Sincronización, Planificación.	Alta
RF5.2	Una falla en un agente delegado no debe bloquear al orquestador ni a los demás agentes (fallas silenciosas).	Alta

ID	Descripción	Prioridad
RF5.3	Debe poder añadirse un nuevo agente/habilidad sin modificar el dominio existente.	Alta
RF5.4	La habilidad de escritura de AgenteCurador sobre Notion/Obsidian debe operar en modo “propone, no aplica solo” hasta que el usuario confirme confianza suficiente (autonomía progresiva).	Alta

RU6 — Tareas y recordatorios

ID	Descripción	Prioridad
RF6.1	El sistema debe sincronizar tareas bidireccionalmente con Todoist.	Alta
RF6.2	El sistema debe sincronizar eventos bidireccionalmente con Google Calendar.	Alta
RF6.3	El sistema debe emitir recordatorios proactivos de tareas próximas a vencer.	Media

RU7 — Acceso móvil

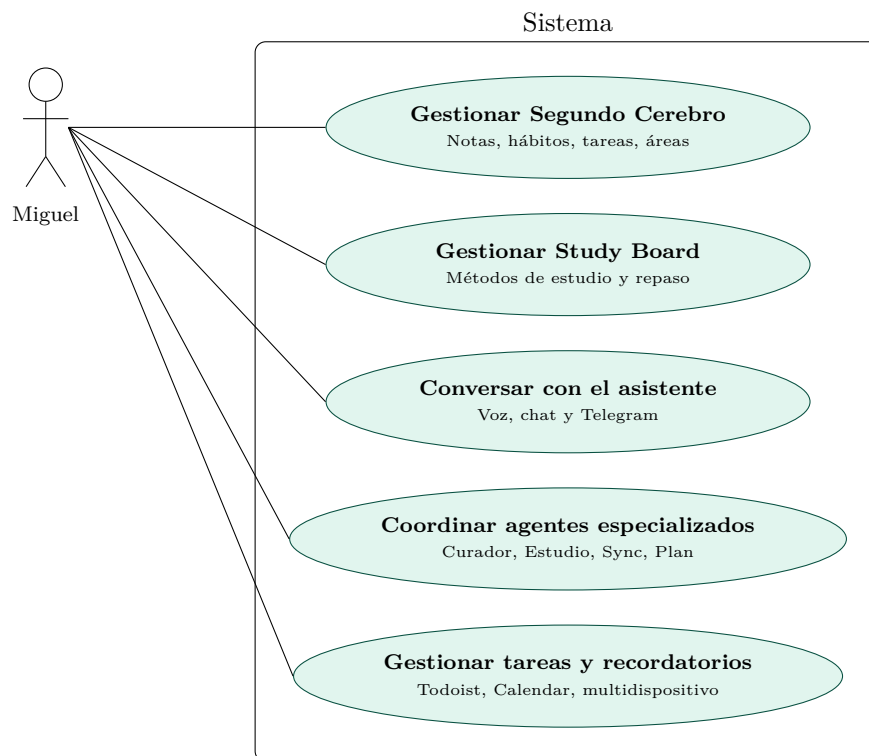
ID	Descripción	Prioridad
RF7.1	El sistema debe exponer el gateway de mensajería nativo de Hermes (Telegram) como interfaz móvil.	Alta
RF7.2	La funcionalidad conversacional en móvil debe ser equivalente a la del HUD, dentro de las limitaciones de Telegram.	Media

4.2 Requerimientos no funcionales

ID	Descripción	Categoría	Prioridad
RNF1	El sistema debe permitir incorporar nuevos agentes, integraciones o módulos sin reestructurar el dominio base (arquitectura hexagonal).	Escalabilidad	Alta
RNF2	Backend Híbrido con Privacidad Local: La inferencia rápida de razonamiento se delega a Gemini Flash (gemini-2.5-flash) protegido obligatoriamente por el middleware local DataMasker (anonimización de PII previa a la red). Los embeddings (nomic-embed-text) y la memoria vectorial (ChromaDB) se mantienen 100 % locales.	Privacidad y Rendimiento	Alta
RNF3	Las respuestas conversacionales deben mantener una latencia inferior a 2 segundos en modo híbrido optimizado.	Rendimiento	Alta
RNF4	Los procesos en segundo plano (sincronización, telemetría) deben tolerar fallos de red sin pérdida de datos.	Disponibilidad	Media
RNF5	La interacción por voz debe soportar español e inglés sin configuración manual de idioma.	Usabilidad	Baja

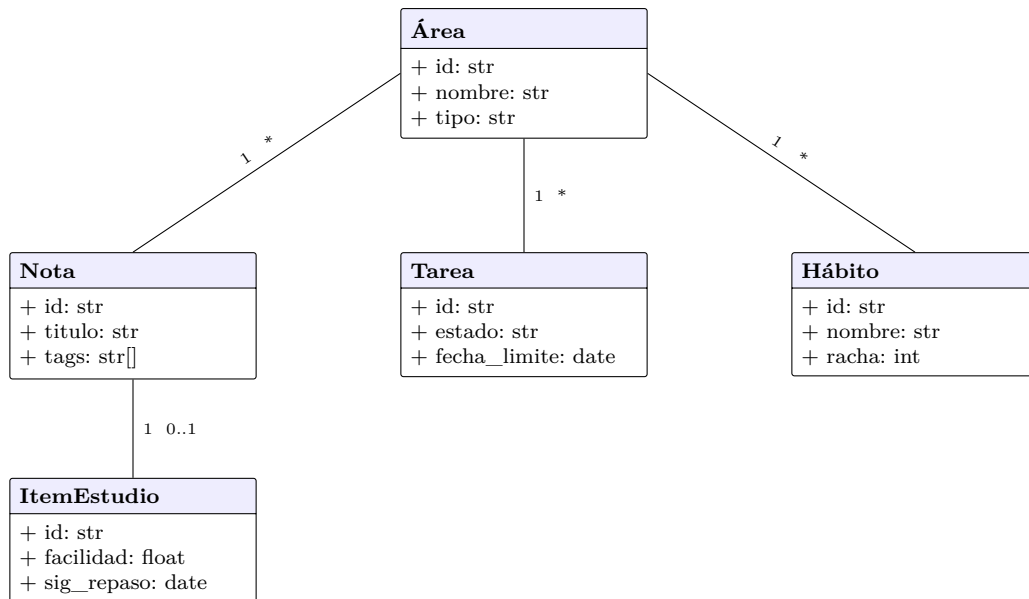
ID	Descripción	Categoría	Prioridad
RNF6	El código debe respetar estrictamente la separación <code>domain/use_cases/infrastructure/entrypoints</code> .	Mantenibilidad	Alta
RNF7	El sistema debe operar en el entorno local (Windows) con persistencia soberana en disco (Obsidian, ChromaDB, JSON).	Portabilidad	Alta

5 Diagrama de casos de uso



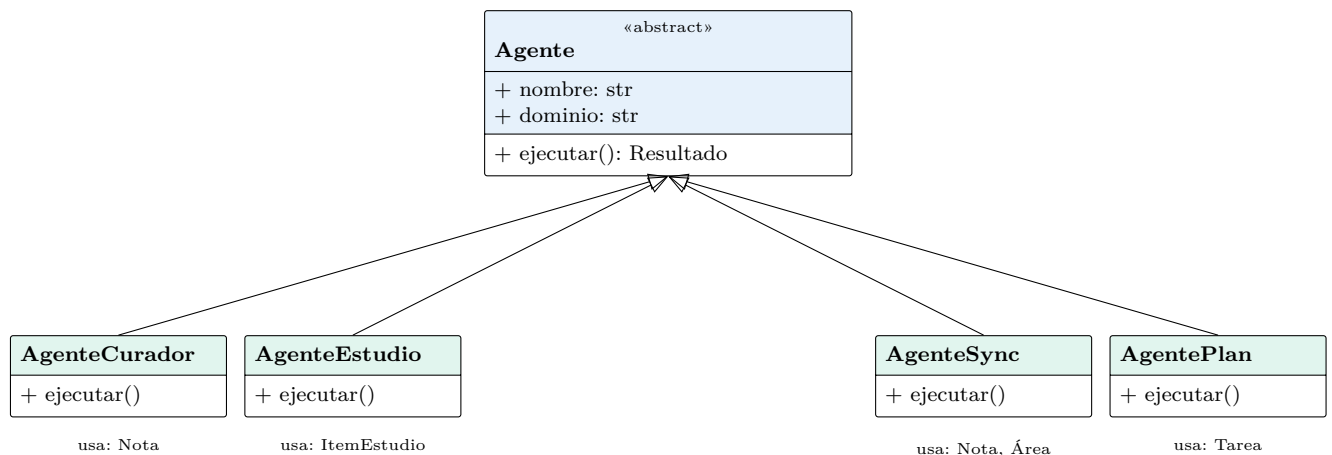
6 Diagrama de clases

6.1 Modelo de dominio



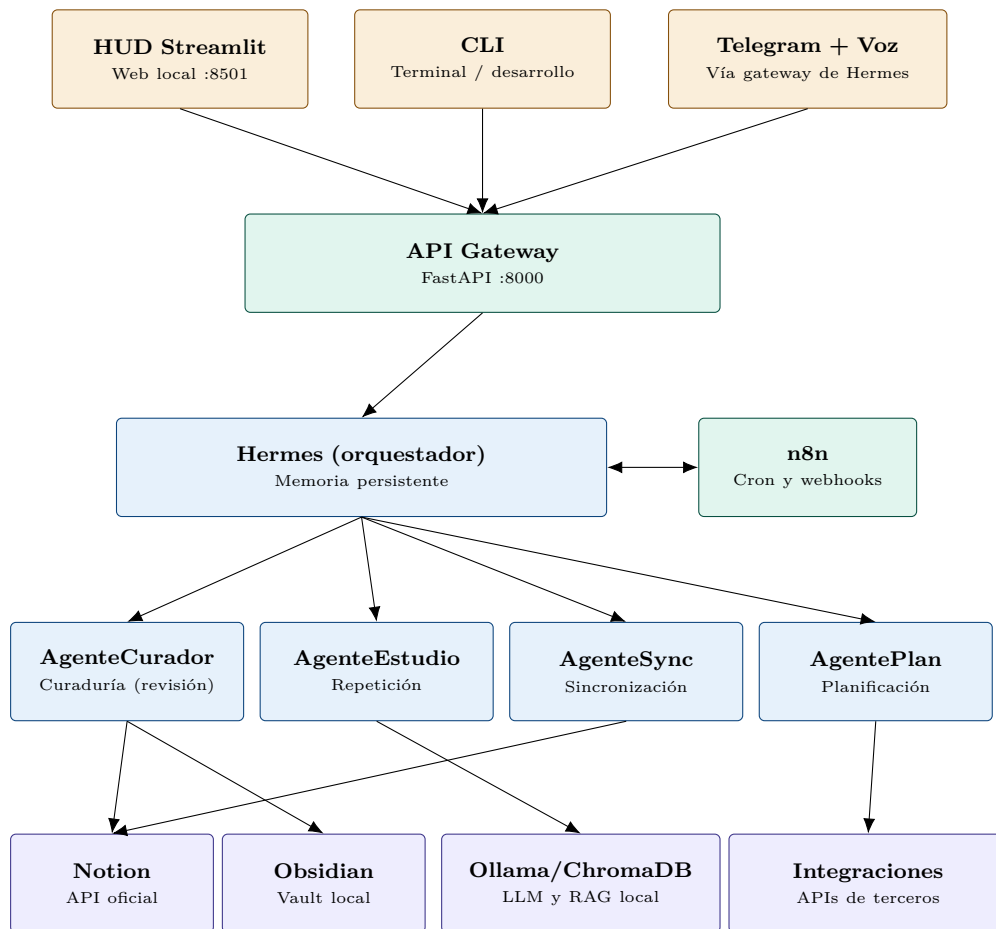
Área (1) contiene (*) muchas notas, tareas y hábitos. Una nota puede generar opcionalmente (0..1) un **ItemEstudio** para repetición espaciada, conectando el Study Board al Segundo Cerebro sin duplicar la base de datos.

6.2 Jerarquía de agentes



Agente es abstracta y vive en `domain/`; cada subtipo sobrescribe `ejecutar()` en `use_cases/` operando sobre una porción distinta del modelo de dominio.

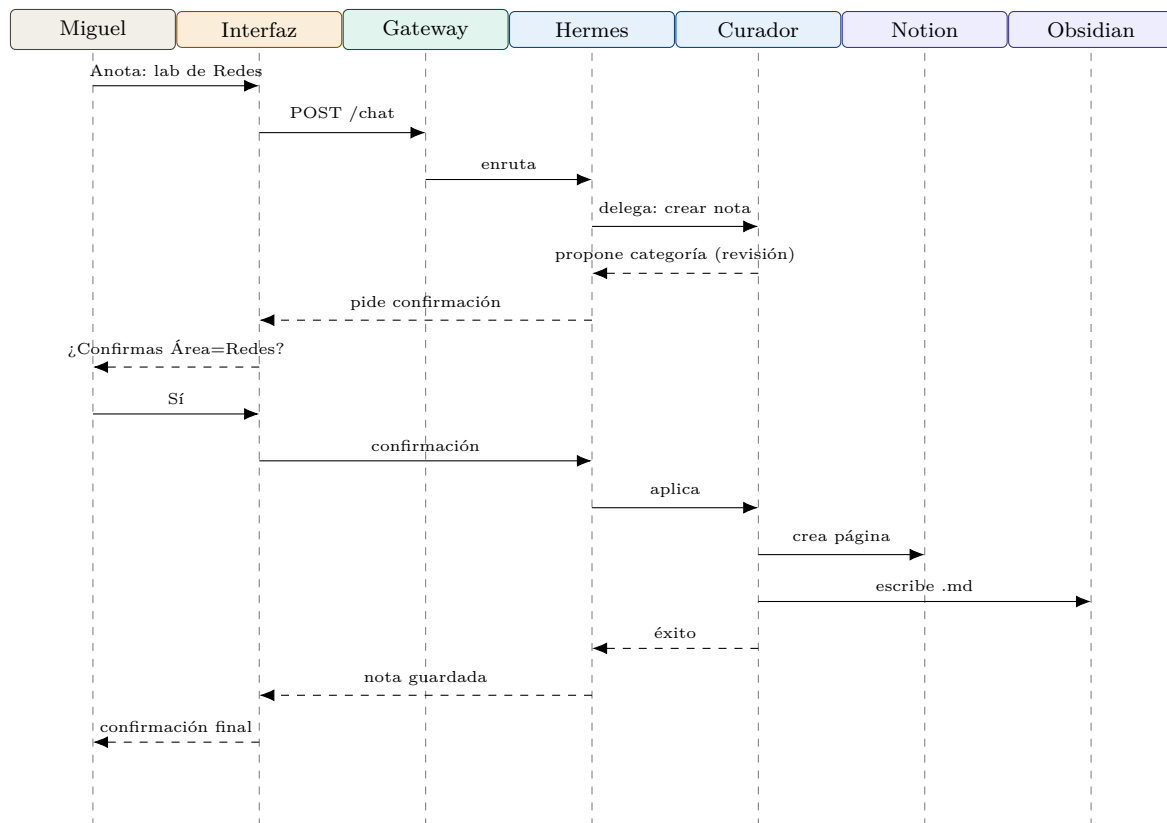
7 Arquitectura de componentes



Integraciones agrupa Todoist, Google Calendar, Google Drive y Canvas LMS bajo un mismo tipo de adaptador (API REST externa). Gemini y NotebookLM se omiten deliberadamente de este diagrama: por RNF2 no forman parte del flujo automático del sistema.

8 Diagrama de secuencia — crear y categorizar una nota

Escenario: Miguel dicta “anota esto: terminé el laboratorio de Redes”.



Líneas sólidas = peticiones; líneas punteadas = respuestas. El intercambio Miguel↔Interfaz↔Hermes (pasos 5–9) es el guardrail de autonomía progresiva (RF5.4): **AgenteCurador** nunca escribe sin que la propuesta pase por Hermes y vuelva a Miguel.

9 Flujos e integración de API

Segundo Cerebro

Servicio	Tipo de acceso	Autenticación	Dirección	Quién lo usa	Disparador
Notion	REST API oficial + Developer Platform	Token de integración	Bidireccional	AgenteCurador	Conversacional + sync periódica
Obsidian	Sistema de archivos local	Ninguna (local)	Bidireccional	AgenteCurador, Al ChromaDB	Al confirmarse una nota
ChromaDB	Vector store local	Ninguna (local)	Escritura/lectura	Hermes, AgenteEstudio	Al modificarse el vault

Study Board

Servicio	Tipo de acceso	Autenticación	Dirección	Quién lo usa	Disparador
Google Drive	REST API	OAuth2	Lectura	AgenteEstudio	On-demand
Canvas LMS	REST API	Token de acceso	Lectura	AgenteEstudio	Sync periódica
Gemini	REST API / SDK	API key	Lectura puntual	AgenteEstudio	Solo bajo pedido explícito (RF2.5)

Tareas y tiempo

Servicio	Tipo de acceso	Autenticación	Dirección	Quién lo usa	Disparador
Todoist	REST API	Token API personal	Bidireccional	AgentePlan	Webhook → n8n → Hermes
Google Calendar	REST API v3	OAuth2	Bidireccional	AgentePlan	Sync periódica

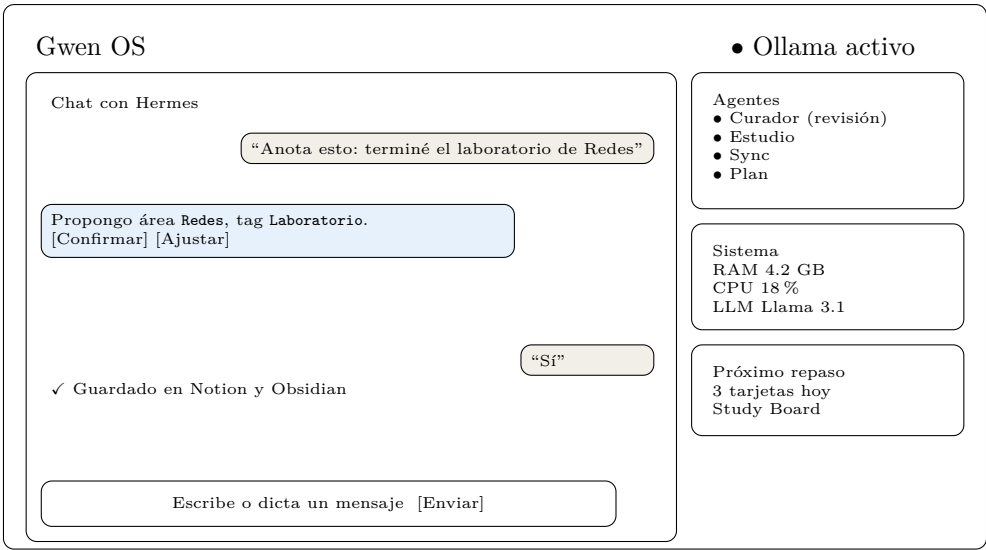
Orquestación interna

Servicio	Tipo de acceso	Autenticación	Dirección	Quién lo usa	Disparador
Hermes	API HTTP local (compatible OpenAI, sobre Ollama)	Ninguna (local)	Bidireccional	API Gateway, n8n	Cada mensaje/evento
n8n	Webhooks + <code>/api/v1/sync</code>	Ninguna (red cerrada)	Bidireccional con Hermes	Todoist, Gateway	Cron / webhook externo
Ollama	API HTTP local :11434	Ninguna (local)	Consulta	Hermes, todos los agentes	Cada razonamiento (<code>keep_alive=30m</code>)

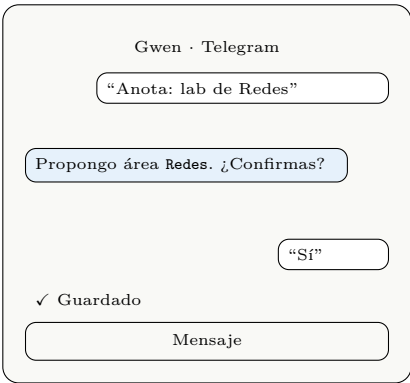
Fuera de esta tabla, a propósito: NotebookLM no tiene API pública, por lo que sigue siendo un paso manual (RF2.4).

10 Wireframes

HUD principal



Interfaz móvil (Telegram)



El guardrail de **AgenteCurador** (RF5.4) debe verse en pantalla como una propuesta con botones de confirmación, tanto en el HUD como en Telegram: si no aparece en la interfaz, el requerimiento no está realmente implementado.

11 Decisiones de arquitectura clave

Decisión	Justificación
Hermes como orquestador único (no perfiles aislados por agente)	Centraliza la memoria persistente y el modelo de “quién es Miguel” en un solo lugar; evita que cada agente construya una versión distinta del usuario. Patrón supervisor/orquestador clásico en sistemas multiagente.

Decisión	Justificación
Backend Híbrido Gemini Flash + DataMasker vs CPU Pura	En hardware de CPU pura (sin GPU NVIDIA dedicada), los LLMs locales de 7B tardaban más de 2 minutos por turno. Al integrar Gemini Flash (<code>gemini-2.5-flash</code>) con <code>thinkingBudget = 0</code> y un middleware de sanitización local (<code>DataMasker</code>), la latencia cayó a 1.5s (>97 % de mejora) manteniendo la privacidad y los embeddings/vectores 100 % locales en disco.
n8n complementa a Hermes, no lo reemplaza	n8n maneja lo determinístico y programado (cron, webhooks); Hermes maneja lo que requiere criterio. Se comunican bidireccionalmente vía <code>AgenteSync</code> .
Autonomía progresiva solo en <code>AgenteCurador</code>	Es el único agente que toca directamente el Segundo Cerebro sin posibilidad trivial de reversión; los demás agentes (<code>Estudio</code> , <code>Sync</code> , <code>Plan</code>) son autónomos desde el día uno por su menor radio de error.
Google Calendar en vez de Notion Calendar como motor de sincronización	Notion Calendar no ofrece sincronización bidireccional nativa y confiable con bases de datos de Notion; Google Calendar sí.
Visualizador 3D Isométrico desacoplado del Gateway	El simulador espacial 3D opera en el frontend mediante WebGL/Three.js consumiendo 0 % de CPU en el servidor backend y ofreciendo telemetría visual inmediata.

12 Conclusiones y estado de implementación

Este documento cierra el ciclo completo de ingeniería de requerimientos para Agent IA: nivel de negocio, nivel de usuario, nivel de software (funcional y no funcional), casos de uso, clases, arquitectura de componentes, secuencia, integración de API y wireframes — todo trazable de punta a punta, cada wireframe apunta a un RF, cada RF a un RU, cada RU al objetivo de negocio.

La plataforma cuenta con el 100 % de sus agentes implementados y probados:

1. **Hermes:** Orquestador semántico con clasificación sub-segundo y máquina de estados.
2. **AgenteCurador:** Curaduría con guardrail de confirmación previa y deduplicación en ChromaDB.
3. **AgentePlan:** Planificador estratégico en cascada y creación rápida de tareas.
4. **AgenteEstudio:** Study Board con flashcards atómicas, notas Cornell y repaso espaciado SM-2.
5. **AgenteSync:** Sincronización bidireccional continua Notion ↔ Obsidian y webhooks para n8n.